

Production Readiness Report

Status: Needs Review. Score: 95/100. Review 2 high-severity findings before production hardening, especially around security and configuration, dependency injection.

SOLUTION

Forge

SCORE

95/100

GRADE

A

STATUS

Needs Review

Analyzed 7/16/2026, 1:38:32 AM

1 projects · 3 open findings · 0 suppressed or accepted

Prepared for engineering review, pull request discussion, and release readiness assessment.

Executive Summary

PRODUCTION READINESS SCORE

95/100

Grade A · Needs Review

This solution needs review before production hardening. .DET found 2 high-severity findings, with risk concentrated in security & configuration, dependency injection reliability.

Score explanation: DotDet calculated the 95/100 readiness score from weighted category scores (Security 84, API 100, EF Core 100, Dependency Injection 96, Architecture 100), finding severity, confidence, suppression state, and release-impact caps across 3 active production root-cause finding(s). Major root causes include SECJWT Weak JWT configuration value; SECJWT Weak JWT configuration value; DI002 IDataProtectionProvider appears unregistered.

PRIMARY CONCERNS

- Security & Configuration
- Dependency Injection Reliability

Category Scores

ARCHITECTURE RULES

100/100

0 findings · No findings detected

SECURITY & CONFIGURATION

84/100

2 findings · Review recommended

EF CORE / MIGRATIONS

100/100

0 findings · No findings detected

DEPENDENCY INJECTION

96/100

1 findings · Minor cleanup

API READINESS

100/100

0 findings · No findings detected

Engineering Assessment Summary

SCORE EXPLANATION

DotDet calculated the 95/100 readiness score from weighted category scores (Security 84, API 100, EF Core 100, Dependency Injection 96, Architecture 100), finding severity, confidence, suppression state, and release-impact caps across 3 active production root-cause finding(s). Major root causes include SECJWT Weak JWT configuration value; SECJWT Weak JWT configuration value; DI002 IDataProtectionProvider appears unregistered.

STRONG AREAS

- Architecture boundaries: 100/100 with no critical/error findings.
- EF Core and migrations: 100/100 with no critical/error findings.
- API reliability: 100/100 with no critical/error findings.
- Dependency injection reliability: 96/100 with no critical/error findings.

HIGHEST RISKS

- Security & Configuration: 2 critical/error findings and score 84/100.
- Dependency Injection: score 96/100 with 1 findings to review.

ARCHITECTURE

- DotDet inferred 1 logical layer(s) across 1 project(s) and 0 project reference(s).

SECURITY

- Security and configuration: Weak JWT configuration value (Error) in solution scope.
- Security and configuration: Weak JWT configuration value (Error) in solution scope.

API READINESS

- API readiness: no major findings were detected.

MAINTAINABILITY

- IDataProtectionProvider appears unregistered can increase operational or change risk if left unresolved.

Architecture

Logical layers, project dependencies, and architecture violations inferred from the analyzed solution.

Presentation
Forge.Api Forge.Api 1 findings

PROJECT GRAPH SUMMARY

No project dependencies were discovered.

Top Open Risks

Open findings with the highest release impact. Suppressed, ignored, false-positive, and accepted-risk findings are listed separately.

Error SECJWT WEAK JWT CONFIGURATION VALUE JWT issuer, audience, or signing key configuration appears weak. Recommended action: Set non-empty issuer/audience values and store a high-entropy signing key outside committed appsettings files.
Error SECJWT WEAK JWT CONFIGURATION VALUE JWT issuer, audience, or signing key configuration appears weak. Recommended action: Set non-empty issuer/audience values and store a high-entropy signing key outside committed appsettings files.
Warning DI002 IDATAPROTECTIONPROVIDER APPEARS UNREGISTERED A constructor-injected application dependency does not have a matching registration. Recommended action: Register IDataProtectionProvider in Program.cs/Startup.cs or in a source-defined IServiceCollection extension method.

Recommended Remediation Roadmap

1 SECJWT: Set non-empty issuer/audience values and store a high-entropy signing key outside committed appsettings files.

2 DI002: Register IDataProtectionProvider in Program.cs/Startup.cs or in a source-defined IServiceCollection extension method.

3 Review high-severity findings with the release owner and document accepted residual risk.

4 Re-run DotDet after remediation and attach the updated report to the pull request or release ticket.

Findings By Category

ARCHITECTURE RULES · 100/100 · 0 FINDINGS

No findings detected for this category.

SECURITY & CONFIGURATION · 84/100 · 2 FINDINGS

SEVERITY	FINDING	LOCATION	RECOMMENDATION
Error	SECJWT Weak JWT configuration value JWT issuer, audience, or signing key configuration appears weak. <pre>"Jwt": { "Issuer": "dotdet-api", "Audience": "dotdet-clients", "Key": "use-a-long-random-value-from-secure-configuration" }</pre>	Solution Forge.SampleShop.Api/appsettings.json:15	Set non-empty issuer/audience values and store a high-entropy signing key outside committed appsettings files. JWT bearer authentication in ASP.NET Core
Error	SECJWT Weak JWT configuration value JWT issuer, audience, or signing key configuration appears weak. <pre>"Jwt": { "Issuer": "dotdet-api", "Audience": "dotdet-clients", "Key": "use-a-long-random-value-from-secure-configuration" }</pre>	Solution RiskyMinimalApi.Api/appsettings.json:9	Set non-empty issuer/audience values and store a high-entropy signing key outside committed appsettings files. JWT bearer authentication in ASP.NET Core

EF CORE / MIGRATIONS · 100/100 · 0 FINDINGS

No findings detected for this category.

DEPENDENCY INJECTION · 96/100 · 1 FINDINGS

SEVERITY	FINDING	LOCATION	RECOMMENDATION
Warning	DI002 IDataProtectionProvider appears unregistered A constructor-injected application dependency does not have a matching registration. • GitHubRepositoryAccessStore: GitHubRepositoryAccessStore injects IDataProtectionProvider at GitHubRepositoryAccessStore.cs:23 (Services/GitHubRepositoryAccessStore.cs:23) • GitHubRepositoryAccessStore: GitHubRepositoryAccessStore injects IDataProtectionProvider at GitHubRepositoryAccessStore.cs:32 (Services/GitHubRepositoryAccessStore.cs:32) <pre>builder.Services.AddScoped<IServiceContract, ServiceImplementation>();</pre>	Forge.Api Services/GitHubRepositoryAccessStore.cs:23	Register IDataProtectionProvider in Program.cs/Startup.cs or in a source-defined IServiceCollection extension method. • ASP.NET Core dependency injection

API READINESS · 100/100 · 0 FINDINGS

No findings detected for this category.

Suppressions And Accepted Risks

These findings remain visible for auditability, but they are separated from the open remediation queue.

No suppressed, ignored, false-positive, or accepted-risk findings.

Rule Explanations

Warning

DI002

Roslyn Syntax Analysis

IDATAPROTECTIONPROVIDER APPEARS UNREGISTERED

Problem: A constructor-injected application dependency does not have a matching registration.

Why it matters: Missing registrations usually surface as startup exceptions or hard-to-trace runtime failures when the container builds the object graph.

Recommended pattern: Every constructor-injected application dependency should be registered in the composition root or a clearly imported module.

Good example:

```
builder.Services.AddScoped<IPaymentGateway, StripePaymentGateway>();  
builder.Services.AddScoped<IOrderWorkflow, OrderWorkflow>();
```

Bad example:

```
public sealed class OrderWorkflow  
{  
    public OrderWorkflow(IPaymentGateway gateway) { }  
}  
// No IPaymentGateway registration found.
```

Microsoft documentation:

- [ASP.NET Core dependency injection](#)

CONNECTION STRING IS STORED IN CONFIGURATION

Problem: A committed configuration file contains a connection string value.

Why it matters: Source-controlled connection strings can leak credentials, database topology, or privileged connection details across environments.

Recommended pattern: Store production secrets outside committed configuration using environment variables, user secrets, or a managed secret store.

Good example:

```
"ConnectionStrings": {  
  "Default": ""  
}  
// DOTNET_ConnectionStrings__Default is supplied by deployment configuration.
```

Bad example:

```
"ConnectionStrings": {  
  "Default": "Server=tcp:prod;User Id=app;Password=P@ssw0rd!"  
}
```

Microsoft documentation:

- [Safe storage of app secrets in development](#)
- [Azure Key Vault configuration provider](#)

Error**SECJWT****Heuristic Analysis**

WEAK JWT CONFIGURATION VALUE

Problem: JWT issuer, audience, or signing key configuration appears weak.

Why it matters: Weak JWT settings can make issued tokens unverifiable, ambiguous across environments, or vulnerable to replay and signing issues.

Recommended pattern: Use non-empty issuer and audience values plus a high-entropy signing key supplied from secure configuration.

Good example:

```
"Jwt": {  
  "Issuer": "https://identity.example.com",  
  "Audience": "dotdet-api",  
  "Key": ""  
}  
// Signing key supplied from a managed secret store.
```

Bad example:

```
"Jwt": {  
  "Issuer": "",  
  "Audience": "",  
  "Key": "dev"  
}
```

Microsoft documentation:

- [JWT bearer authentication in ASP.NET Core](#)

Warning**SECSECRET****Heuristic Analysis**

POSSIBLE SECRET STORED IN CONFIGURATION

Problem: A configuration key looks sensitive and contains a non-placeholder value.

Why it matters: Secrets committed into configuration files are easy to copy, leak, or reuse in ways that weaken production controls.

Recommended pattern: Keep API keys, tokens, and passwords in managed secret storage rather than source-controlled files.

Microsoft documentation:

- [Safe storage of app secrets in development](#)